

山东大学
网络空间安全学院

《网络安全》
实验报告

学生姓名	江李阳 (学号:202400460042), 王健一 (学号:202400460039), 邱泽辉 (学号:202400460046)
指导教师	郭山清
学院	网络空间安全学院
专业班级	密码科学与技术 2024 级一班
完成时间	2026 年 9 月 10 日

1 实验目标

- 通过实验操作和网络流量分析，深入理解交换机和路由器的工作原理。
- 掌握子网划分和子网掩码的概念和应用，理解子网掩码在网络通信中的重要性。
- 熟练使用 tcpdump 工具，掌握获取和分析网络流量的技能。
- 为后续章节的学习和深入理解网络原理奠定基础，提升解决实际网络问题的能力。

2 实验原理

2.1 ARP

ARP(Address Resolution Protocol, 地址解析协议)，用于将 IP 地址解析为 MAC 地址。因为以太网通信时除了知道 IP 地址外，还需要知道下一跳的 MAC 地址，才能发送以太网帧。比如 A 与 B 通信时，A 希望给 B 发送以太网帧，就必须知道 B 的 MAC 地址，如果 A,B 不在同一个网段，那么 A 需要知道网关的 MAC 地址。

当发送 IP 数据包时，主机先查询自己的 ARP 缓存表，如果已经有目标 IP 和 MAC 的映射，则直接使用该映射发送以太网帧 (IP 数据包封装在以太网帧中)。如果没有，则需要先构造 ARP 请求，具体过程如下：

- ARP 请求时发送广播帧，源 MAC 地址为 A 的 MAC 地址，目的 MAC 地址为 FF:FF:FF:FF:FF:FF 即广播帧；以太网类型为 0x0806 表示 ARP；操作码为 1，表示请求；源 IP 地址为 A 的 IP 地址；ARP 报文中的目标硬件地址字段为全 0(因为此时还不知道目标 MAC 地址)，目的 IP 地址为要解析的 IP 地址，如 B 的 IP 或网关 IP。
- 交换机收到广播帧后会向同一广播域内的其他端口泛洪，因此同一局域网内的主机都会收到该 ARP 请求的广播帧，但只有目标 IP 匹配的主机才会处理该请求，其他主机直接丢弃。
- 目标主机 (或网关) 发现所问的就是自己时，首先会更新自身的 ARP 缓存，记录请求方的 IP 和 MAC 地址；接着构造 ARP 响应，操作码为 2，该回复的以太网帧的目的 MAC 是请求方 MAC，即以单播回复。
- A 收到 ARP 响应后将对应的下一跳 IP 与下一跳的 MAC 对应关系记录下来，写入 ARP 缓存中。

2.2 ICMP

ICMP(Internet Control Message Protocol, 互联网控制报文协议) 属于网络层协议，但封装在 IP 包中，IPv4 首部中的协议号为 1 表示上层协议为 ICMP。它不是面向应用的数据传输协议，主要用于报告错误、网络诊断和路由控制等。常见类型包括：

- 查询类报文：Echo Request 的 Type 为 8，用于请求目标主机回应；Echo Reply 的 Type 为 0，用于回应 Echo Request。

差错类报文及其常见用途如下，其中 Redirect 报文会在子网掩码配置错误的实验中出现：

表 1 常见 ICMP 差错类报文

类型	名称	用途
3	Destination Unreachable	目标不可达
4	Source Quench	源抑制，已废弃
5	Redirect	重定向，告知主机有更好的下一跳
11	Time Exceeded	超时，TTL 减到 0 或分片重组超时
12	Parameter Problem	IP 头部字段问题

Type 3 和 Type 11 报文中常见的 Code 字段含义如下：

表 2 ICMP Type 3 的常见 Code

Code	含义
0	网络不可达
1	主机不可达
2	协议不可达
3	端口不可达
4	需要分片但 DF 位置位
5	源路由失败

2.3 Ping

Ping 的本质是发送 ICMP Echo Request，等待对方回复 ICMP Echo Reply，计算往返时间 RTT。Ping 并不使用 TCP/UDP，因此没有端口，而是直接使用 ICMP。Ping 的完整流程如下 (假设 A Ping B)：

表 3 ICMP Type 11 的常见 Code

Code	含义
0	TTL 超时
1	分片重组超时

- 解析目标 IP: A 执行 `ping test.com`, 此时 Ping 的目标如果是域名, 系统先会做 DNS 解析, 得到对应的 IP; 如果 Ping 的是 IP, 则直接使用。
- 构造 ICMP 报文 (Type=8, 类型为 Echo Request): 封装进 IP 包, 加上 IP 头部信息, 包括源 IP(A 的 IP 地址) 和目的 IP(B 的 IP 地址), 协议号为 1 表示 ICMP。
- 封装 IP 包: 发送前必须知道下一跳 MAC, 如果是 B 在相同子网下, 下一跳就是 B, ARP 解析出 B 的 MAC 地址 (查 ARP 缓存或构造 ARP 请求); 如果 B 与 A 不在同一子网, 即跨网段了, 则 ARP 解析网关的 MAC 地址。最终封装成以太网帧并发出。
- 网络转发: 若 B 与 A 在相同子网, 则由交换机根据 MAC 和端口映射关系直接转发; 若不在同一子网, 则由路由器转发, 经过路由器时会去掉原来的以太网帧头, 将其中的 IP 包部分字段做修改 (如 TTL 值减一、重新计算校验和等), 查询路由表确认下一跳 IP 地址以及对应 MAC 地址, 重新封装以太网帧 (如果 TTL 减为 0, 路由器直接丢包, 并返回 ICMP Time Exceeded)。
- 收到 ICMP 请求: 当 B 收到 IP 包后, 检查发现目的 IP 的确是自身且协议是 ICMP(类型 8, Echo Request), 然后 B 将构造 Echo Reply 并将该回复发送给 A(该过程同样要经历查 ARP 并发送给下一跳)。
- 收到 ICMP 回复: 当 A 收到 Echo Reply 后会先检查 ICMP 校验和、检查序号等参数与之前的请求相匹配, 然后用收到时间减去发送时的时间计算 RTT。

3 实验设置

4 实验一：使用 tcpdump 分析 ICMP 流量

4.1 实验配置

本实验使用 VMware 搭建实验环境, Client(Attacker) 通过 OpenWRT 路由器与其他主机进行通信。Client 使用的网卡为 `ens33`, 其地址为 192.168.3.10, OpenWRT 内网接口地址为 192.168.3.1, Victim 地址为 192.168.3.200, External Server 地址为 192.168.4.105。实验开始时 Client 和 Victim 使用 /24 掩码, 第三部分实验再将 Client 的掩码修改为 /28。

在 Client 上使用如下命令清空 ARP 缓存并捕获 ARP 和 ICMP 报文：

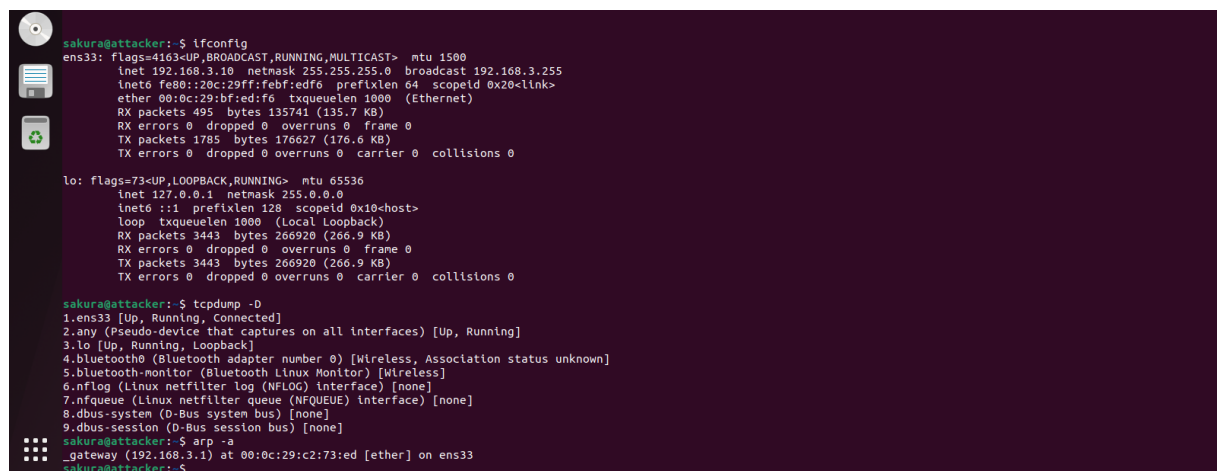
Listing 1 Client 上的 tcpdump 抓包命令

```
1 sudo ip neigh flush dev ens33
2 sudo tcpdump -i ens33 -n -e 'arp or icmp'
```

其中，`-i ens33` 表示在 `ens33` 网卡上抓包，`-n` 表示不进行地址解析，`-e` 表示显示以太网帧头中的 MAC 地址。

4.2 子网内通信

首先登录 Client，查看当前的网络接口、可监听的网卡、ARP 缓存表等如下图：然后在



```
sakura@attacker:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.3.10  netmask 255.255.255.0  broadcast 192.168.3.255
    inet6 fe80::20c:29ff:feb7:edf6  prefixlen 64  scopeid 0x20<link>
    ether 00:0c:29:bf:ed:f6  txqueuelen 1000  (Ethernet)
    RX packets 495  bytes 135741 (135.7 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 1785  bytes 176627 (176.6 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 3443  bytes 266920 (266.9 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 3443  bytes 266920 (266.9 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

sakura@attacker:~$ tcpdump -D
1.ens33 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.bluetooth0 (Bluetooth adapter number 0) [Wireless, Association status unknown]
5.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
6.nflog (Linux netfilter log (NFLOG) interface) [none]
7.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
8.dbus-system (D-Bus system bus) [none]
9.dbus-session (D-Bus session bus) [none]
sakura@attacker:~$ arp -a
_ _ _ _ _
_gateway (192.168.3.1) at 00:0c:29:c2:73:ed [ether] on ens33
sakura@attacker:~$
```

图 1 client 的网络信息

Client 上清空 ARP 缓存，在另一个终端执行（注意此前 Client 已经开启监听 `ens33` 网卡）：

Listing 2 子网内通信测试

```
1 ping -c 4 192.168.3.200
```

由于 `192.168.3.10` 和 `192.168.3.200` 均属于 `192.168.3.0/24`，Client 判断目标主机与自己处于同一子网内，因此不会先将数据发送给网关，而是直接广播 ARP 请求，询问 `192.168.3.200` 对应的 MAC 地址。Victim 收到请求后单播返回 ARP 响应，Client 随后将 ICMP Echo Request 直接发送给 Victim。

从抓包结果可以看到，ARP 请求的源 MAC 地址为 Client 的 MAC 地址 `00:0c:29:bf:ed:f6`，目的以太网 MAC 地址为广播地址；ARP 响应的源 MAC 地址为 Victim 的 MAC 地址 `00:0c:29:2c:12:a8`。之后出现源 IP 为 `192.168.3.10`、目的 IP 为 `192.168.3.200` 的 ICMP Echo Request，以及方向相反的 ICMP Echo Reply。

```
17:57:29.860189 00:0c:29:bf:ed:f6 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), Length 42: Request who-has 192.168.3.200 tell 192.168.3.10, length 28
17:57:29.861937 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype ARP (0x0806), Length 60: Reply 192.168.3.200 is-at 00:0c:29:2c:12:a8, length 46
17:57:29.861951 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype IPv4 (0x0800), Length 98: 192.168.3.10 > 192.168.3.200: ICMP echo request, id 10, seq 1, length 64
17:57:29.862681 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype IPv4 (0x0800), Length 98: 192.168.3.200 > 192.168.3.10: ICMP echo reply, id 10, seq 1, length 64
17:57:30.420800 00:0c:29:c2:73:ed > 00:0c:29:bf:ed:f6, ethertype ARP (0x0806), Length 60: Request who-has 192.168.3.10 tell 192.168.3.1, length 46
17:57:30.420823 00:0c:29:bf:ed:f6 > 00:0c:29:c2:73:ed, ethertype ARP (0x0806), Length 42: Reply 192.168.3.10 is-at 00:0c:29:bf:ed:f6, length 28
17:57:30.861899 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype IPv4 (0x0800), Length 98: 192.168.3.10 > 192.168.3.200: ICMP echo request, id 10, seq 2, length 64
17:57:30.865399 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype IPv4 (0x0800), Length 98: 192.168.3.200 > 192.168.3.10: ICMP echo reply, id 10, seq 2, length 64
17:57:30.911990 00:0c:29:c2:73:ed > 00:0c:29:2c:12:a8, ethertype ARP (0x0806), Length 60: Request who-has 192.168.3.200 tell 192.168.3.1, length 46
17:57:30.916239 00:0c:29:2c:12:a8 > 00:0c:29:c2:73:ed, ethertype ARP (0x0806), Length 60: Reply 192.168.3.200 is-at 00:0c:29:2c:12:a8, length 46
17:57:31.145757 00:0c:29:2c:12:a8 > 00:0c:29:c2:73:ed, ethertype ARP (0x0806), Length 60: Request who-has 192.168.3.1 tell 192.168.3.200, length 46
17:57:31.146111 00:0c:29:c2:73:ed > 00:0c:29:2c:12:a8, ethertype ARP (0x0806), Length 60: Reply 192.168.3.1 is-at 00:0c:29:c2:73:ed, length 46
17:57:31.864198 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype IPv4 (0x0800), Length 98: 192.168.3.10 > 192.168.3.200: ICMP echo request, id 10, seq 3, length 64
17:57:31.865510 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype IPv4 (0x0800), Length 98: 192.168.3.200 > 192.168.3.10: ICMP echo reply, id 10, seq 3, length 64
17:57:32.866128 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype IPv4 (0x0800), Length 98: 192.168.3.10 > 192.168.3.200: ICMP echo request, id 10, seq 4, length 64
17:57:32.866571 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype IPv4 (0x0800), Length 98: 192.168.3.200 > 192.168.3.10: ICMP echo reply, id 10, seq 4, length 64
17:57:35.243110 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype ARP (0x0806), Length 60: Request who-has 192.168.3.10 tell 192.168.3.200, length 46
17:57:35.243137 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype ARP (0x0806), Length 42: Reply 192.168.3.10 is-at 00:0c:29:bf:ed:f6, length 28
^C
32 packets captured
32 packets received by filter
0 packets dropped by kernel

sakura@attacker: ~
sakura@attacker:~$ ping -c 4 192.168.3.200
PING 192.168.3.200 (192.168.3.200) 56(84) bytes of data:
64 bytes from 192.168.3.200: icmp_seq=1 ttl=64 time=2.85 ms
64 bytes from 192.168.3.200: icmp_seq=2 ttl=64 time=3.48 ms
64 bytes from 192.168.3.200: icmp_seq=3 ttl=64 time=1.34 ms
64 bytes from 192.168.3.200: icmp_seq=4 ttl=64 time=0.481 ms

--- 192.168.3.200 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.481/2.039/3.484/1.189 ms
sakura@attacker:~$
```

图 2 子网内 Client 与 Victim 通信的 tcpdump 抓包结果

4.3 跨子网通信

在保持 Client 地址不变的情况下，清空 ARP 缓存后访问 External Server:

Listing 3 跨子网通信测试

```
1 sudo ip neigh flush dev ens33
2 ping -c 4 192.168.4.105
```

192.168.4.105 不属于 Client 所在的 192.168.3.0/24 网段，因此 Client 需要将数据包交给默认网关 192.168.3.1。Client 首先通过 ARP 请求获取网关的 MAC 地址，OpenWRT 返回 ARP 响应。之后，Client 发送的以太网帧目的 MAC 地址是 OpenWRT 内网接口的 MAC 地址 00:0c:29:c2:73:ed，但 IP 层的目的地址仍然是 External Server 的 192.168.4.105。

在 Client 的 ens33 网卡上抓包时，看到的跨网段 ICMP 报文为 192.168.3.10 与 192.168.4.105 之间的 Echo Request 和 Echo Reply，而以太网帧的下一跳 MAC 地址是 OpenWRT 的 MAC 地址。这说明路由器负责三层转发，并在不同网段重新封装以太网帧。

4.4 子网掩码错误

第三部分实验按照实验指导书的要求，将 Client 的子网掩码由/24 修改为/28。修改后 Client 的地址和路由信息为:

Listing 4 修改 Client 子网掩码并验证

```
1 sudo nano /etc/netplan/01-network-manager-all.yaml
2 sudo netplan generate
```

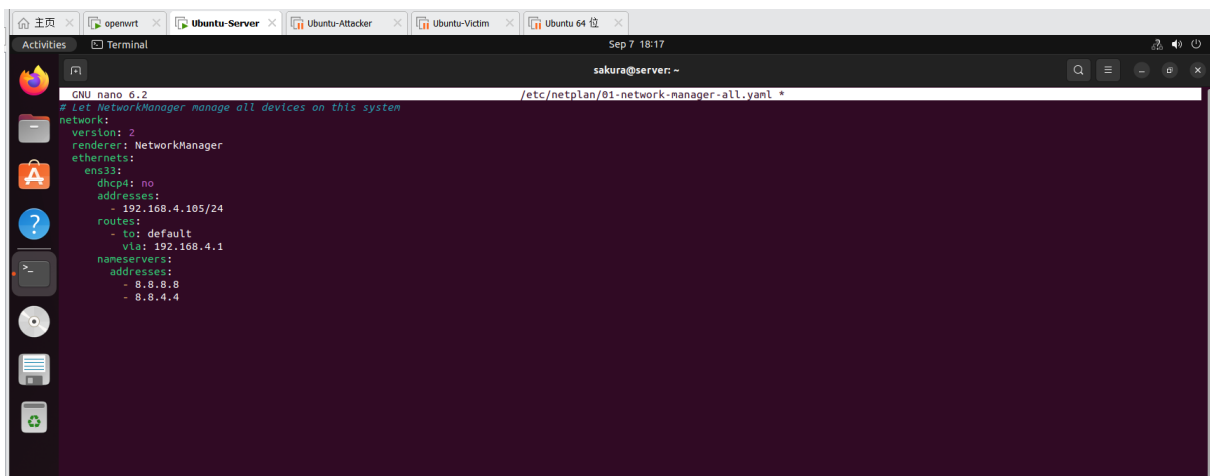


图 3 External Server 的网络配置

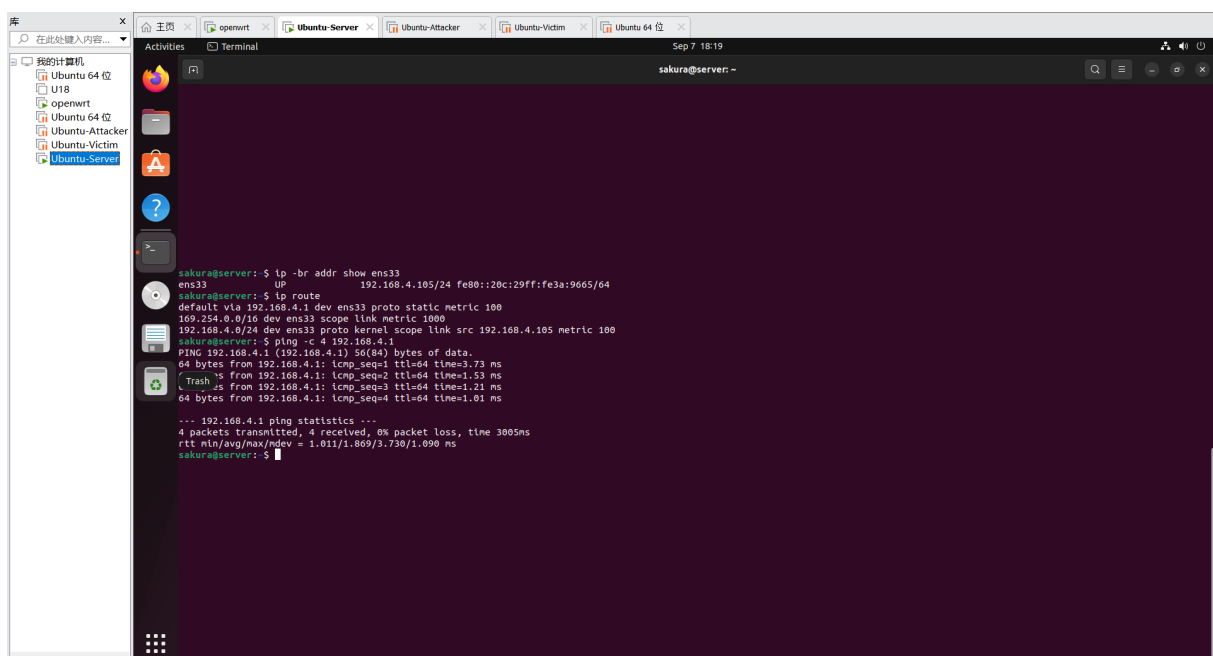


图 4 跨子网通信前的配置验证

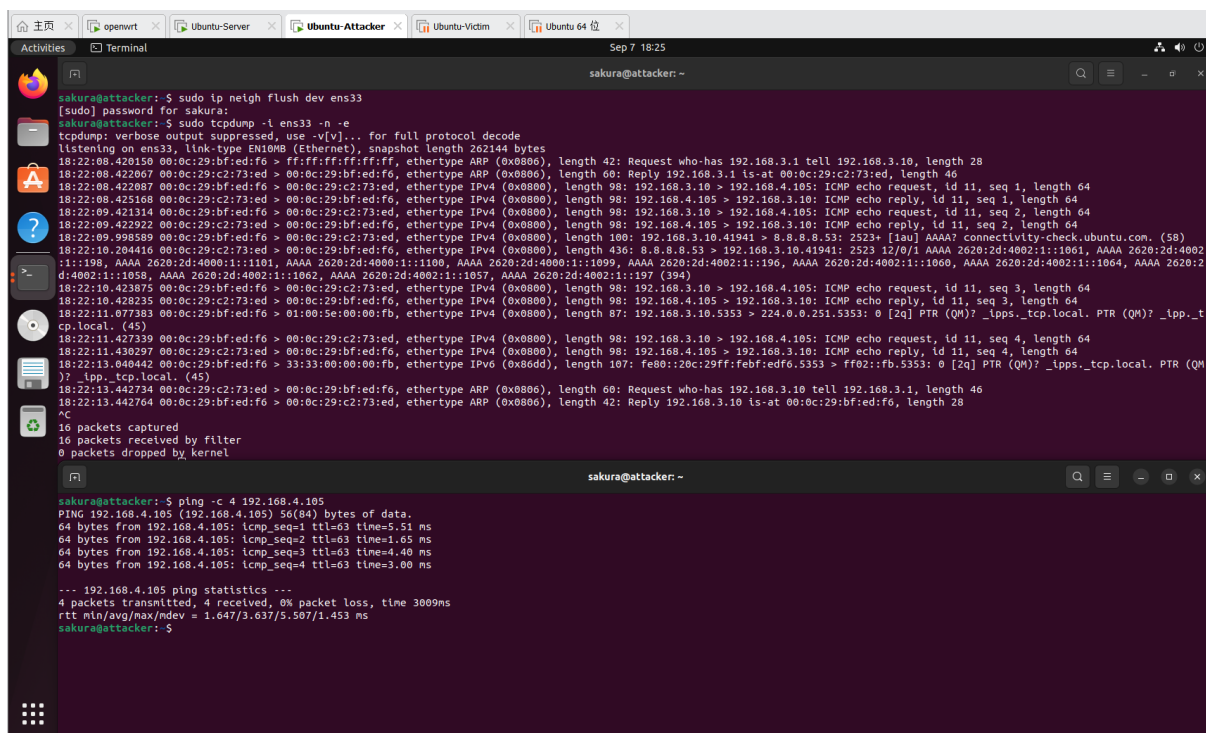


图 5 Client 访问 External Server 的实验结果

```
3 sudo netplan try
4 ip -br addr show ens33
5 ip route
```

修改后,Client 的地址变为 192.168.3.10/28,直连网段为 192.168.3.0/28。虽然 Victim 的地址仍然是 192.168.3.200, 并且二者实际上连接在同一个二层网络中, 但 Client 根据/28 掩码判断 192.168.3.200 不属于本地子网, 因此首先将 ICMP 报文发送给默认网关 192.168.3.1。

由于 OpenWRT 发现目标 192.168.3.200 与报文进入的接口相同, 路由器向 Client 返回了 ICMP Redirect 报文, 并提示新的下一跳为 192.168.3.200。Client 收到重定向后重新进行 ARP 解析, 获取 Victim 的 MAC 地址, 之后直接向 Victim 发送 ICMP Echo Request。因此, 抓包中依次可以观察到: Client 与网关之间的 ARP 请求和响应、ICMP Redirect、Client 对 Victim 的 ARP 请求和响应, 以及最终的 ICMP Echo Request 和 Echo Reply。

从图中可以看到, ping 过程中出现了“Redirect Host”提示, 同时也收到了 Victim 的 Echo Reply。终端最后显示 3 个数据包中收到 2 个回复, 并统计了 2 个错误。这里的错误是 Linux 的 ping 程序将 ICMP Redirect 作为错误报文统计的结果, 并不表示 Victim 完全不可达; tcpdump 已经证明重定向前后均存在 ICMP Echo Reply。

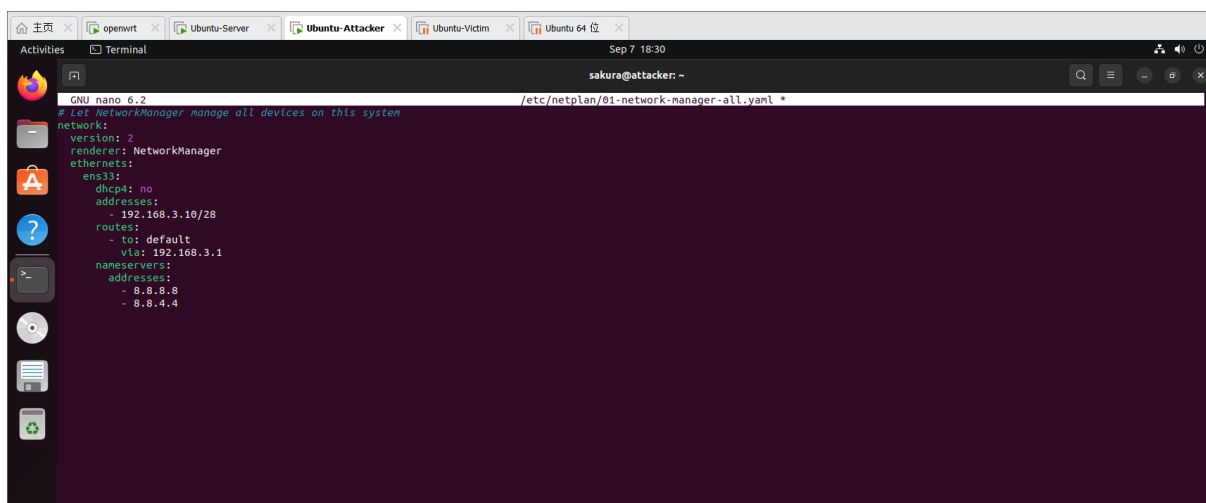


图 6 修改 Client 子网掩码为/28

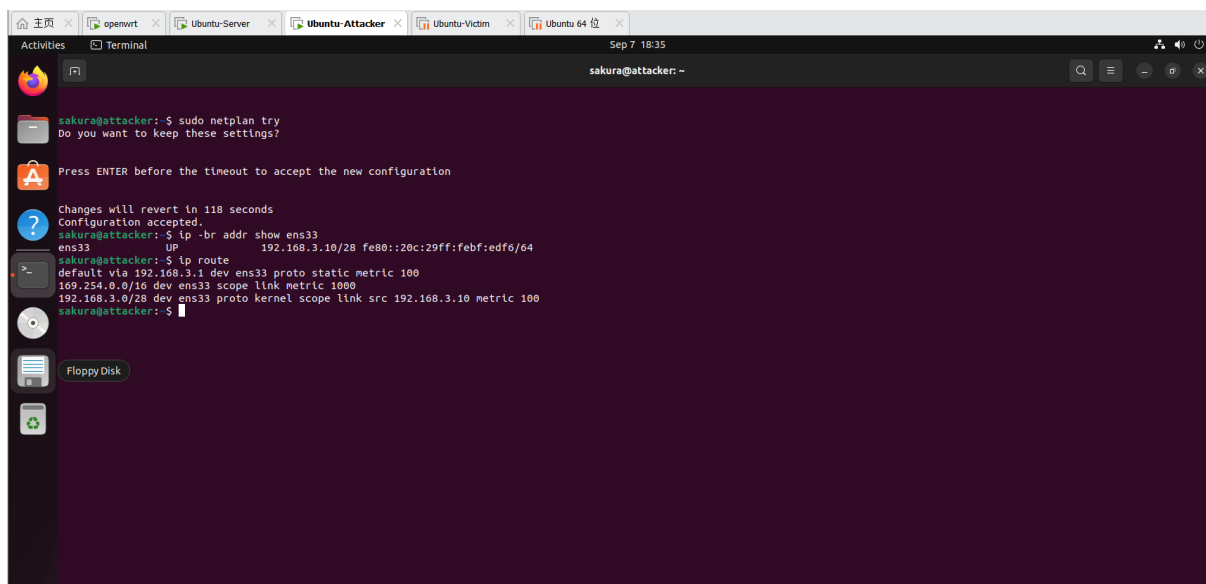


图 7 /28 掩码配置验证

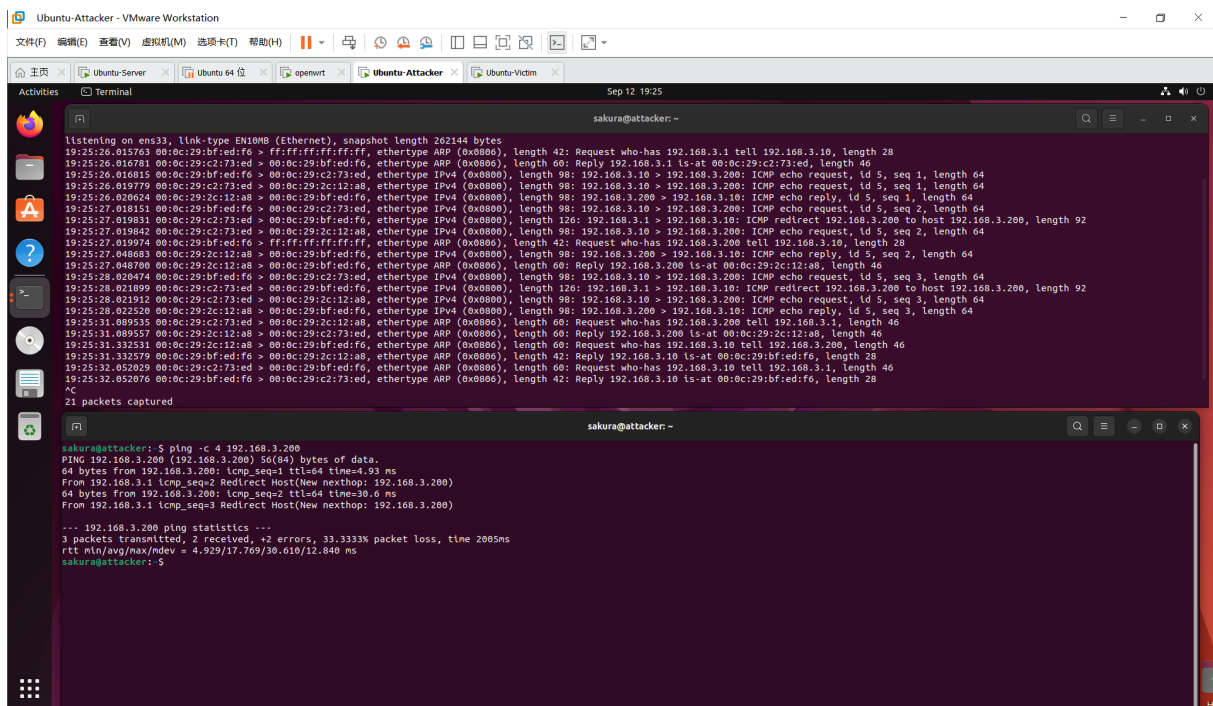


图 8 子网掩码错误时的 tcpdump 与 ping 结果

4.5 流量分析记录表

表4按照实验指导书中的表 1-1-1 填写。表中的 MAC 地址是从 Client 的 ens33 网卡抓包结果中读取的；因此在跨子网通信中，Client 看到的下一跳 MAC 是 OpenWRT 内网接口的 MAC，而不是 External Server 在另一个网段中的 MAC。

表 4 实验一流量分析记录表

操作	ARP 请求		ARP 响应		ICMP ECHO 请求		ICMP ECHO 响应	
	源 MAC	目标 MAC	源 MAC	目标 MAC	源 IP	目标 IP	源 IP	目标 IP
子网内	00:0c:29:bf:ed:f6	ff:ff:ff:ff:ff:ff	00:0c:29:2c:12:a8	00:0c:29:bf:ed:f6	192.168.3.10	192.168.3.200	192.168.3.200	192.168.3.10
跨子网	00:0c:29:bf:ed:f6	ff:ff:ff:ff:ff:ff	00:0c:29:c2:73:ed	00:0c:29:bf:ed:f6	192.168.3.10	192.168.4.105	192.168.4.105	192.168.3.10
掩码错（初始）	00:0c:29:bf:ed:f6	ff:ff:ff:ff:ff:ff	00:0c:29:c2:73:ed	00:0c:29:bf:ed:f6	192.168.3.10	192.168.3.200	192.168.3.200	192.168.3.10
掩码错（重定向后）	00:0c:29:bf:ed:f6	ff:ff:ff:ff:ff:ff	00:0c:29:2c:12:a8	00:0c:29:bf:ed:f6	192.168.3.10	192.168.3.200	192.168.3.200	192.168.3.10

根据实际的 Ping 结果应当指出，“掩码错”对应两个阶段：初始阶段 Client 向网关 192.168.3.1 发出 ARP 请求，ARP 响应来自 OpenWRT；收到 ICMP Redirect 后，Client 再次发出 ARP 请求，ARP 响应来自 Victim。两个阶段的 ICMP 源、目的 IP 仍然是 192.168.3.10 和 192.168.3.200，变化的是以太网帧的下一跳 MAC 地址。

4.6 三次操作的结果差异及原因分析

根据表4，三次操作的主要差异在于 ARP 解析的对象、以太网帧的下一跳 MAC 地址以及是否经过路由器转发，具体原因是 Client 的子网掩码改变了它对目标地址所属网络

的判断。

- 子网内通信时，Client 和 Victim 都属于 192.168.3.0/24。Client 直接 ARP 解析 Victim 的 MAC 地址，之后以太网帧由交换机直接转发，ICMP Echo Request 和 Echo Reply 的 IP 地址分别为 192.168.3.10 和 192.168.3.200，过程中不需要经过 OpenWRT 进行三层路由。
- 跨子网通信时，External Server 不属于 Client 所在的 192.168.3.0/24 网段。Client 不能直接解析 External Server 的 MAC 地址，而是 ARP 解析默认网关 192.168.3.1 的 MAC 地址，由 OpenWRT 进行三层转发。因此在 Client 的网卡上，IP 层的目的地址是 External Server 的地址，但以太网帧的目的 MAC 地址是 OpenWRT 内网接口的 MAC 地址。
- 子网掩码错误时，Client 使用 192.168.3.10/28，将 192.168.3.200 错误判断为远端地址，所以首先 ARP 解析网关并把 ICMP 报文发送给 OpenWRT。OpenWRT 发现目标与报文进入同一个内网接口，于是返回 ICMP Redirect，并告知 Client 可以把 192.168.3.200 作为新的下一跳。Client 随后重新 ARP 解析 Victim 并直接发送数据。因此，该操作同时出现了网关 ARP、ICMP Redirect 和 Victim ARP，形成了与前两次操作不同的报文序列。

由此可见，三次操作中 ICMP 报文的 IP 通信端点并不一定改变，但子网掩码会影响主机的路由判断，进而决定 ARP 请求的目标、以太网帧的下一跳以及是否触发路由器的重定向。

4.7 实验小结

本实验通过 tcpdump 观察了 ARP 和 ICMP 报文在不同网络配置下的变化。子网内通信时，Client 直接解析 Victim 的 MAC 地址并发送数据；跨子网通信时，Client 解析默认网关的 MAC 地址，由 OpenWRT 完成三层转发；子网掩码错误时，Client 错误地把同一二层网络中的 Victim 判断为远端主机，先将数据交给网关，并收到 ICMP Redirect。由此可以看出，子网掩码不仅决定主机对目标地址的判断，也会直接影响 ARP 解析对象、下一跳选择和数据包的转发过程。

- 5 实验二：使用 Wireshark 分析 Web 访问过程流量
- 6 实验三：使用 Scapy 构造 ICMP Echo Request
- 7 实验讨论
- 8 参考资料
- 9 附录：关键代码